

SAFE SECURITY AUDIT

Rapport d'audit de sécurité non destructif

Élément	Valeur
Date de l'audit	21 septembre 2026
Branche et commit	main / cbd8417
État Git	Arbre de travail propre ; aucune modification de l'application
Périmètre	API Laravel active, interface React/TypeScript, ancien serveur Express, dépendances et configuration Render
Environnement dynamique	Environnement local uniquement ; PostgreSQL Render et données de production non interrogés

Conclusion exécutive : audit terminé — aucune modification effectuée. Un défaut d'autorisation SEC-001 est confirmé dans la route de changement du mot de passe d'une entreprise. Les vérifications de production PostgreSQL/Render restent non exécutées.

1. État général

- **Architecture** : Laravel et Eloquent constituent l'API active ; PostgreSQL est la base cible de production ; Express reste conservé pour la compatibilité et le retour arrière.
- **Environnement local** : l'API a répondu avec le serveur PHP intégré et une base SQLite locale. Cette différence empêche de conclure à la parité PostgreSQL de production.
- **Configuration Render inspectée** : mode production, débogage désactivé, PostgreSQL, cookies sécurisés et disque persistant prévu pour les fichiers numériques.
- **Git** : branche main, commit cbd8417, branche distante alignée et différence vide au moment du contrôle.
- **Consigne appliquée** : aucun mot de passe, compte, paiement, notification de paiement, fichier dangereux, secret, donnée, migration, configuration Render ou déploiement n'a été modifié.

Contrôles réalisés

Contrôle	Résultat	Interprétation
Analyse des dépendances	0 critique, 1 élevée, 3 modérées, 2 faibles	Présence de risques signalés ; aucune mise à jour
Analyse statique du code	Analyse incomplète, 0 résultat remonté	Ne prouve pas l'absence de vulnérabilité
Analyse HoundDog	0 vulnérabilité	Aucune vulnérabilité remontée par cet outil
Validations antérieures	234 tests Laravel, 1 720 assertions, 147 tests interface, vérification des types, compilation et syntaxe PHP	Résultats conservés ; non relancés durant cette phase
Requêtes HTTP locales	GET, HEAD et OPTIONS sûrs ; notification de paiement sans signature rejetée	Aucune mutation métier

2. Résumé

Les observations applicatives sont séparées des alertes de dépendances. Une alerte de dépendance confirme la présence d'un paquet signalé dans l'arbre des paquets, mais ne démontre pas à elle seule une exploitation dans le chemin d'exécution Laravel.

Indicateur	Compte
Signalements structurés	10 : 4 observations applicatives et 6 alertes de dépendances
CRITIQUES	0
ÉLEVÉS	2 : SEC-001 et 1 alerte js-yaml
MOYENS	6 : SEC-002, SEC-003, politique CORS et 3 alertes modérées
FAIBLES	2 alertes de dépendances
CONFIRMÉS	2 : SEC-001 et présence des alertes de dépendances
POTENTIELS	3 : SEC-002, SEC-003 et politique CORS
TEST NON EXÉCUTÉ	Production Render/PostgreSQL, changement de mot de passe, paiements, téléversement et réjouement de notification signée

Note de lecture : les six alertes de dépendances sont comptées individuellement ; les contrôles applicatifs sont regroupés par sujet.

3. Constats détaillés

SEC-001 — Changement du mot de passe administrateur insuffisamment restreint

Gravité : **ÉLEVÉE**

Statut : CONFIRMÉ

Zone : PATCH /api/auth/company-password — AuthController::updateCompanyPassword

Preuve : La route applique seulement le middleware maximus.auth. Pour tout acteur qui n'est pas maximus_admin, le contrôleur prend automatiquement son entreprise, recherche le premier compte actif company_admin, remplace son hash et invalide ses sessions. Aucun contrôle explicite du rôle company_admin ni Policy dédiée n'est appliqué avant cette opération.

Reproduction sûre : Aucune requête de changement n'a été envoyée. La preuve statique suffit : un acteur employee authentifié pourrait appeler la route sans companyId et atteindre la mise à jour du compte administrateur de son entreprise.

Impact potentiel : Un compte employé compromis pourrait provoquer une prise de contrôle ou un déni d'accès du compte administrateur entreprise. La réauthentification n'est pas demandée par cette route.

Correction proposée : Ajouter une autorisation serveur explicite pour maximus_admin ou company_admin autorisé, vérifier le périmètre, demander une réauthentification récente et ajouter un test négatif employee/sector_manager qui vérifie 403 sans modifier de données.

Risque de correction : Risque de bloquer un parcours d'administration existant ; cartographier les rôles attendus et tester sur une base PostgreSQL dédiée.

SEC-002 — Profil d'installation public trop détaillé

Gravité : MOYENNE

Statut : POTENTIEL

Zone : GET /api/installation — InstallationContext::publicProfile

Preuve : La réponse publique contient le mode, companyOnly, installationId, canonicalUrl/loginUrl, les états ready/adminLoginEnabled/registrationEnabled et, en mode isolé, une synthèse de synchronisation. En mode central local, aucun secret ni identifiant d'entreprise n'a été observé.

Reproduction sûre : GET local sans authentification : réponse 200 et profil JSON ; aucune écriture. La production n'a pas été interrogée.

Impact potentiel : Reconnaissance de l'installation, de l'entreprise et de son état opérationnel. Aucun secret n'a été observé localement ; le statut reste potentiel.

Correction proposée : Limiter le profil aux informations indispensables au démarrage public ; retirer installationId, erreurs de synchronisation détaillées et identifiants d'entreprise si l'interface n'en a pas besoin.

Risque de correction : Risque de casser l'amorçage de l'interface ou les connexions personnalisées si un champ est utilisé implicitement.

SEC-003 — Point de santé public révélant l'état interne

Gravité : MOYENNE

Statut : POTENTIEL

Zone : GET /api/healthz

Preuve : La réponse locale contient ok=true, database=true, status=OPERATIONAL, applicationVersion=unknown, syncProtocolVersion=2 et checkedAt. Le point de santé est aussi utilisé par Render et ne renvoie ni trace ni chaîne de connexion.

Reproduction sûre : GET et HEAD locaux sans authentification : réponse 200. OPTIONS annonce GET et HEAD. Aucun accès à la production.

Impact potentiel : Divulcation limitée de l'état et des versions, utile à la reconnaissance. L'usage pour la surveillance est légitime ; la gravité reste potentielle.

Correction proposée : Conserver l'état nécessaire à Render mais réduire la réponse publique à un minimum ; réserver les détails de version et de protocole au diagnostic authentifié.

Risque de correction : Risque de casser le contrat attendu par Render ou un outil de surveillance.

SEC-004 — Politique CORS trop large à revalider

Gravité : MOYENNE

Statut : POTENTIEL

Zone : Réponses API locales GET, HEAD et OPTIONS

Preuve : Les réponses observées portent Access-Control-Allow-Origin: *. L'interface utilise credentials: include ; avec un wildcard, les navigateurs refusent normalement la lecture avec identifiants, mais la politique reste trop large pour une API authentifiée par cookie.

Reproduction sûre : GET, HEAD et OPTIONS locaux avec curl, sans cookie et sans mutation. Aucun scénario intersite avec identifiants n'a été lancé.

Impact potentiel : Accès intersite public élargi et risque de régression si une future configuration autorise les identifiants ou un autre mode d'authentification.

Correction proposée : Définir explicitement les origines nécessaires pour l'ERP, les entreprises et les boutiques ; désactiver le wildcard pour les routes privées.

Risque de correction : Risque de bloquer une boutique ou une installation légitime si la liste des domaines est incomplète.

Alerte de dépendances — DEP-001

Statut : CONFIRMÉ pour la présence dans l'arbre des paquets ; exploitabilité dans le chemin d'exécution non confirmée.

L'analyse a trouvé 1 alerte élevée, 3 modérées et 2 faibles. Le paquet explicitement identifié comme élevé est **js-yaml 4.3.1**. Les autres paquets explicitement signalés sont **esbuild 0.18.20/0.27.3**, **nanoid 3.3.16** et **qs 6.15.3**. Deux alertes faibles supplémentaires sont conservées dans le compte de l'analyse sans inventer leur nom à partir de la synthèse disponible.

- **Preuve** : résultat de l'analyse des dépendances ; aucune dépendance n'a été mise à jour.
- **Impact** : il dépend de l'utilisation effective des chemins concernés, surtout pour les outils de compilation ou les dépendances indirectes ; aucune exploitation n'a été démontrée.
- **Correction proposée** : examiner les chemins directs et indirects, appliquer les versions corrigées après validation, puis relancer l'analyse, les tests, la vérification des types et la compilation.
- **Risque** : modification de la résolution pnpm, incompatibilité de compilation ou variation du paquet livré ; aucune correction automatique pendant cet audit.

4. Requêtes HTTP non destructives

Méthode	Adresse locale	Authentification	Résultat et conclusion
GET	/api/healthz	Non requise	200 ; état OPERATIONAL et database=true ; informations de santé exposées
HEAD	/api/healthz	Non requise	200 ; mêmes en-têtes de sécurité et absence de mise en cache
OPTIONS	/api/healthz	Non requise	200 ; GET et HEAD autorisés ; wildcard CORS observé
GET	/api/installation	Non requise	200 ; profil public ; aucun secret observé localement
HEAD	/api/installation	Non requise	200 ; endpoint public confirmé
OPTIONS	/api/auth/login	Non requise	200 ; POST annoncé ; aucune mutation
GET	/api/auth/session	Non requise	200 ; user=null ; session anonyme correctement gérée
GET	/api/diagnostics/health	Jeton Bearer requis	401 ; jeton de diagnostic absent
HEAD	/api/ecommerce/bootstrap	Session requise	401 ; absence de session correctement bloquée
POST	/api/payments/diamanopay/webhook	Signature requise	401 ; signature invalide rejetée avant traitement

Ces requêtes ont été exécutées sur 127.0.0.1:8080, sans cookie de session, sans jeton secret et sans contenu susceptible de modifier une donnée. La notification de paiement non signée a été rejetée avant tout traitement métier.

5. Protections correctement observées

- Le mot de passe est haché côté serveur et les sessions de l'administrateur sont invalidées après une mise à jour autorisée ; le défaut SEC-001 concerne l'autorisation de l'appelant.
- Les sessions utilisent un jeton aléatoire stocké sous forme de hash SHA-256, une expiration serveur de huit heures et un cookie HTTP-only, SameSite=Lax et sécurisé en production.
- Les routes privées refusent une session absente ; les comptes inactifs et les entreprises inactives sont rejetés.
- La résolution d'entreprise ne laisse pas un acteur non-MAXIMUS élargir son companyId ; les requêtes e-commerce consultées bornent les ressources par entreprise.
- Les jetons d'installation sont comparés via token_hash ; les routes d'installation et de diagnostic possèdent des middleware dédiés.
- Les notifications DiamanoPay exigent une signature HMAC vérifiée avec hash_equals ; le traitement des paiements est conçu pour être idempotent.
- Les téléversements d'images observés vérifient le type de fichier, les extensions autorisées et une taille maximale ; aucun fichier dangereux n'a été envoyé.

- Les réponses portent une politique CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy et HSTS en production.
- La configuration Render inspectée désactive le débogage, utilise PostgreSQL et prévoit un stockage persistant sous /var/data.

6. Tests non exécutés

- Production Render et PostgreSQL : **TEST NON EXÉCUTÉ** — risque d'interagir avec des données réelles.
- Changement réel de mot de passe SEC-001 : **TEST NON EXÉCUTÉ** — action sur un compte sensible.
- Création ou confirmation de paiement, retrait et jeu d'une notification signée : **TEST NON EXÉCUTÉ** — risque financier et effet de bord externe.
- Téléversement multipart, traversée de chemin et exécution serveur : **TEST NON EXÉCUTÉ** en production ; l'audit est resté statique pour cette partie.
- Isolation active entre deux entreprises, matrice complète des permissions et parité Laravel/PostgreSQL : **TEST NON EXÉCUTÉ** de bout en bout.
- Analyse statique du code : **TEST NON EXÉCUTÉ** au sens probant — l'outil a retourné une analyse incomplète avec zéro résultat.

7. Corrections proposées et contrôles après correction

- **Priorité 1** : corriger SEC-001 par un contrôle de rôle et de Policy côté serveur, une réauthentification pour le compte sensible et des tests négatifs sans mutation réelle.
- **Priorité 2** : réduire les réponses publiques de /api/installation et /api/healthz aux champs strictement nécessaires.
- **Priorité 3** : remplacer le wildcard CORS par une liste d'origines explicite après inventaire des domaines.
- **Priorité 4** : traiter les paquets signalés, en commençant par js-yaml 4.3.1, sans mise à jour automatique ; valider la compilation et les chemins réellement livrés.
- **Avant publication** : relancer les tests Laravel sur PostgreSQL dédié, les tests d'isolation et de rôles, la vérification des types, la compilation, l'analyse des dépendances, l'analyse statique complète et HoundDog.

8. Conclusion

AUDIT TERMINÉ — AUCUNE MODIFICATION EFFECTUÉE.

Le défaut SEC-001 est le résultat le plus important : la route de changement du mot de passe entreprise doit être corrigée avant toute nouvelle exposition à des utilisateurs non administrateurs. Les autres points sont des risques potentiels ou des alertes de dépendances à qualifier sans déduction abusive. La production Render/PostgreSQL n'a pas été altérée et ne peut pas être déclarée conforme sur la seule base de cet audit local.

Ce rapport ne contient aucune valeur de secret, aucun mot de passe, aucun jeton, aucune donnée financière ni donnée métier.